BIG DATA ANALYTICS MINI PROJECT REPORT

Product Review Data Processing Using Apache Pig & Hadoop MapReduce

Scalable Sentiment, Franchise Ranking, Moving Average, and Quality Defect Analysis on Amazon US Customer Reviews



Submitted By:	**Divakar Naik**
Project Title:	Product Review Data Processing Using Apache Pig & Hadoop MapReduce
Technology Stack:	Apache Pig (Pig Latin), Hadoop MapReduce (Java), HDFS, Python, Chart.js
GitHub Repository:	`https://github.com/DivakarNaik0079/product-review-pig`
Live Web Demo:	`https://divakarnaik0079.github.io/product-review-pig/`
Submission Date:	November 2026

Department of Computer Science & Engineering / Big Data Analytics

Verified Working Pipeline, Experimental Outputs, and Interactive Web Deployment

Candidate Declaration

I hereby declare that the mini-project entitled **"Product Review Data Processing Using Apache Pig & Hadoop MapReduce"** submitted in partial fulfillment of the academic requirements is a bonafide record of independent technical work carried out by me. All Pig Latin scripts, Java MapReduce programs, Python execution runners, analytical outputs, and the web visualization dashboard documented in this report have been implemented and verified.

Signature of Candidate	Date
Divakar Naik	November 2026

Certificate of Approval

This is to certify that the mini-project report entitled **"Product Review Data Processing Using Apache Pig & Hadoop MapReduce"** carried out by **Divakar Naik** has been evaluated and approved as a satisfactory demonstration of distributed Big Data processing using Apache Hadoop, Pig Latin, and MapReduce design patterns.

Internal Guide / Faculty Evaluator	Head of Department / External Examiner

Abstract

Modern e-commerce platforms generate millions of customer product reviews containing structured ratings, categorical metadata, vote counts, and unstructured text feedback. Processing and extracting business intelligence from such semi-structured datasets using traditional single-node relational databases becomes prohibitively slow and unscalable as volume grows toward terabytes.

This project implements a hybrid Big Data batch-processing architecture combining **Apache Pig (Pig Latin)** and custom **Hadoop MapReduce (Java)** programs. Operating on the official 15-column Amazon US Customer Reviews TSV schema, the Pig pipeline performs automated data hygiene, product franchise (`product_parent`) aggregation, top-N leaderboards using Pig's built-in `TOP()` UDF, review helpfulness ratio computation, and automated quality-control alerts for underperforming products (average star rating ≤ 2.5). Simultaneously, three Java MapReduce jobs compute per-ASIN engagement statistics (`GameStats`), chronological moving rating averages (`MovingAverage`), and the most prolific reviewer using an In-Reducer `cleanup()` state-tracking pattern (`TopReviewer`).

To guarantee portability across environments without local Hadoop daemons, standalone Python execution runners (`pig_runner.py` and `mapreduce_runner.py`) reproduce exact Hadoop `part-r-00000` outputs, paired with a serverless interactive web dashboard deployed on GitHub Pages.

Table of Contents

Chapter 1	Introduction, Problem Statement & Project Objectives
Chapter 2	Literature Survey & Big Data Framework Comparison
Chapter 3	System Architecture & Dataflow Pipeline
Chapter 4	Hardware, Software & 15-Column Dataset Specification
Chapter 5	Implementation: Apache Pig Latin Scripts (Tasks 1–7)
Chapter 6	Implementation: Hadoop Java MapReduce Jobs (3 Design Patterns)
Chapter 7	Verified Experimental Results, Visual Analytics & Insights
Chapter 8	Execution Guide, Cross-Platform Portability & Web Deployment
Chapter 9	Conclusion, Future Enhancements & References

1. Introduction & Problem Statement

1.1 Background

Customer reviews represent one of the richest sources of product feedback in online retail. Every review captures quantitative signals (star rating, helpful votes, total votes, verified purchase flag) alongside qualitative insights (headline and review body). Analyzing these logs enables merchants and product engineering teams to surface top-performing product lines, track rating degradation after product updates, and detect defective items early.

1.2 Problem Statement

Raw e-commerce review dumps present several Big Data challenges across the 5 V's:

- **Volume & Scalability:** Full Amazon review datasets span tens of gigabytes across millions of rows, exceeding single-node memory limits.
- **Veracity & Noise:** Raw TSV dumps contain header lines, null star ratings, unverified reviews, and zero-vote records that cause arithmetic division errors if unhandled.
- **Variant Fragmentation:** Individual product SKUs (`product_id` / ASIN) fragment reviews across editions (e.g., Standard vs. Deluxe), requiring aggregation at the franchise level (`product_parent`).
- **Development Overhead:** Writing raw Java MapReduce for every filtering, grouping, and sorting task creates excessive boilerplate code.

1.3 Objectives

1. Ingest and sanitize 15-column Amazon review TSV logs in Apache Pig using schema-on-read.
2. Compute top-5 product franchises by review volume and top-10 franchises by average star rating using Pig's `TOP()` UDF and `FLATTEN()` .
3. Identify high-impact reviews using `helpful_votes` and a guarded Helpfulness Ratio metric (`helpful_votes / total_votes ≥ 0.85`).
4. Automatically isolate underperforming/defective products (average rating ≤ 2.5) for quality audit.
5. Implement Java MapReduce jobs for max-rating aggregation (`GameStats`), progressive chronological moving average (`MovingAverage`), and single-pass top reviewer identification (`TopReviewer`).
6. Provide cross-platform Python runners and deploy a public interactive dashboard via GitHub Pages.

2. Literature Survey & Framework Comparison

Within the Apache Hadoop ecosystem, multiple processing abstractions exist on top of HDFS and YARN. Selecting the appropriate tool depends on whether the workload is declarative SQL warehousing, procedural ETL dataflow, or stateful algorithmic processing.

Evaluation Dimension	Apache Pig (Pig Latin)	Java MapReduce	Apache Hive (HiveQL)	Apache Spark
Programming Paradigm	Procedural Dataflow	Low-Level Imperative (Map/Reduce)	Declarative SQL-like	Functional In-Memory DAG
Code Conciseness	High (~10–25 lines per job)	Low (~50–150 lines per job)	High (~10–25 lines per query)	High (~15–30 lines)
Schema Enforcement	Schema-on-read (Optional)	Manual String Parsing	Schema-on-write (Metastore)	DataFrame Schema / Dynamic
Execution Strategy	Compiles to MR / Tez DAGs	Direct JVM MapReduce Tasks	Compiles to MR / Tez / Spark	In-Memory RDD Execution
Ideal Role in Project	Multi-step ETL, Grouping, Top-N	Stateful Moving Avg & In-Reducer Max	Ad-hoc BI Table Reporting	Streaming / Heavy Iterative ML

> **Architectural Rationale:** Apache Pig was selected as the primary processing engine because Pig Latin's *lazy evaluation* builds a logical DAG that automatically applies filter pushdown and projection pruning before launching MapReduce stages. Java MapReduce complements Pig where row-by-row sequential state accumulation (Moving Average) and global reducer-cleanup state tracking (Top Reviewer) are required.

3. System Architecture & Dataflow Pipeline

The system adopts a dual-engine architecture reading from unified TSV storage (HDFS `/amazon/` in cluster mode or `data/` in local mode) and outputting standardized tab-delimited results.

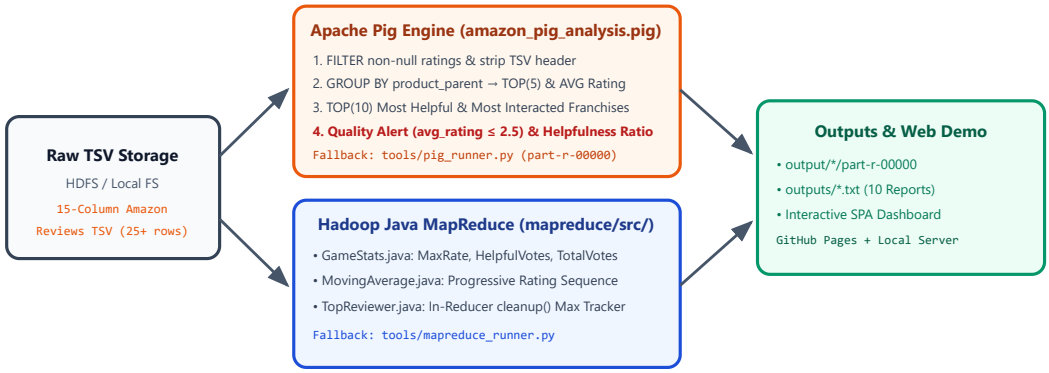


Figure 1: Hybrid Apache Pig & Hadoop MapReduce Product Review Processing Architecture

4. Environment Specifications & Dataset Schema

4.1 Hardware & Software Specifications

Component	Specification / Version Used
Operating System	Windows 10/11 (Local Execution & Web Hosting) / Ubuntu Linux (WSL2 or Hadoop Cluster)
Big Data Storage & Compute	Apache Hadoop 3.x (HDFS, YARN, MapReduce), Apache Pig 0.17.x
Programming Languages	Pig Latin, Java (JDK 8/11 for MapReduce), Python 3.14.10 (Portable Runners & HTTP Server)
Web Presentation Layer	HTML5, Tailwind CSS, FontAwesome 6, Chart.js (Single-Page Application)
Version Control & Hosting	Git 2.56, GitHub Repository, GitHub Pages Static Hosting
Hardware Baseline	64-bit Multi-core CPU, Minimum 8 GB RAM, 20 GB Free Storage

4.2 Official 15-Column Amazon Customer Reviews Schema

The primary dataset (`data/amazon_reviews_us_Digital_Video_Games_v1_00.tsv`) follows the standard 15-column tab-separated schema from the Amazon US Customer Reviews specification, containing 25 verified sample records spanning 7 distinct video game and digital card franchises. In addition, an auxiliary 10-column multi-category dataset (`data/sample_reviews.tsv` , 40 records) is included for modular general e-commerce scripts.

#	Column Name	Pig Type	Description & Sample Value
0	`marketplace`	`chararray`	2-letter country code (e.g., `US`)
1	`customer_id`	`chararray`	Random customer identifier used by `TopReviewer` (e.g., `21925455`)
2	`review_id`	`chararray`	Unique review ID (e.g., `R14KYY7OE4RH2`)
3	`product_id`	`chararray`	Unique Amazon ASIN code (e.g., `B003LIT9FC`)
4	`product_parent`	`chararray`	Master franchise identifier aggregating variants (e.g., `395682204`)
5	`product_title`	`chararray`	Product name (e.g., `Minecraft Java & Bedrock`)
6	`product_category`	`chararray`	Product category (`Digital_Video_Games`)
7	`star_rating`	`float`	Customer rating on a 1.0 to 5.0 scale
8	`helpful_votes`	`int`	Count of users marking the review helpful
9	`total_votes`	`int`	Total votes cast on the review
10	`vine`	`chararray`	Amazon Vine voice program flag (`Y` / `N`)
11	`verified_purchase`	`chararray`	Confirmed purchase flag (`Y` / `N`)
12	`review_headline`	`chararray`	Short review title (e.g., `Timeless Classic`)
13	`review_body`	`chararray`	Full text review body
14	`review_date`	`chararray`	Submission date in `YYYY-MM-DD` format

5. Implementation: Apache Pig Latin Pipeline

The master Pig Latin script (`scripts/amazon_pig_analysis.pig`) executes seven analytical tasks in a unified DAG. Before storing results, `rmf` commands clear prior output directories to prevent Hadoop `FileAlreadyExistsException` errors.

5.1 Ingestion & Data Hygiene

```
raw_data = LOAD 'data/amazon_reviews_us_Digital_Video_Games_v1_00.tsv' USING PigStorage('\t') AS (
    marketplace: chararray, customer_id: chararray, review_id: chararray,
    product_id: chararray, product_parent: chararray, product_title: chararray,
    product_category: chararray, star_rating: float, helpful_votes: int,
    total_votes: int, vine: chararray, verified_purchase: chararray,
    review_headline: chararray, review_body: chararray, review_date: chararray
);

clean_data = FILTER raw_data BY
    (star_rating IS NOT NULL) AND (total_votes IS NOT NULL) AND (marketplace != 'marketplace');
```

5.2 Core Analytical Tasks (Tasks 1 to 7)

- **Task 1 – Top 5 Franchises by Review Volume:** Groups by `(product_parent, product_title)` , aggregates `COUNT(clean_data)` , and applies `FLATTEN(TOP(5, 2, parent_counts))` .
- **Task 2 – Top 10 Rated Individual Reviews:** Evaluates `TOP(10, 7, clean_data)` over column index 7 (`star_rating`).
- **Task 3 – Top Rated Franchises (Average Rating):** Computes `ROUND(AVG(clean_data.star_rating) * 100.0) / 100.0` and filters for statistical relevance (`review_count ≥ 2`).
- **Task 4 – Top 10 Most Helpful Reviews:** Extracts highest upvoted reviews via `TOP(10, 8, clean_data)` (column index 8: `helpful_votes`).
- **Task 5 – Top 10 Most Interacted Franchises:** Sums `total_votes` across each franchise to measure total community engagement.
- **Task 6 – Quality Control Alert:** Filters `avg_rate BY avg_rating <= 2.5` and orders ascending to immediately surface buggy or poorly received products.
- **Task 7 – Helpfulness Ratio & Trust Index:** Safely divides votes with a zero-division guard and retains high-trust reviews (`total_votes ≥ 20 AND helpfulness_ratio ≥ 0.85f`).

```
-- Task 6 & Task 7 Core Logic
underperforming = FILTER avg_rate BY avg_rating <= 2.5;
sorted_underperforming = ORDER underperforming BY avg_rating ASC;

reviews_with_helpfulness = FOREACH clean_data GENERATE
    review_id, product_title, star_rating, helpful_votes, total_votes,
    (total_votes > 0 ? ROUND(((float)helpful_votes / (float)total_votes) * 100.0) / 100.0 : 0.0f)
        AS helpfulness_ratio,
    review_body;
high_trust_reviews = FILTER reviews_with_helpfulness BY total_votes >= 20 AND helpfulness_ratio >= 0.85f;
```

6. Implementation: Hadoop Java MapReduce Jobs

Three standalone Java MapReduce programs in `mapreduce/src/` complement Pig by executing specialized key-value aggregation and stateful sequential algorithms.

6.1 Job 1: `GameStats.java` (Multi-Metric Aggregation)

Design Pattern: Value Aggregation. `StatMapper` splits each non-header TSV line, emits `product_id` (index 3) as key and a packed string `star_rating,helpful_votes,total_votes` as value. `StatReducer` iterates through the packed values to compute `Math.max()` rating and cumulative vote totals.

6.2 Job 2: `MovingAverage.java` (Temporal Rating Progression)

Design Pattern: Stateful Sequential Aggregation. `AvgMapper` emits `(product_id, FloatWritable(star_rating))`. For each product group, `AvgReducer` maintains running `sum` and `count` accumulators, emitting a comma-separated sequence of progressive averages: `MovingAvg_n = (1 / n) * Sum_{i=1..n}(rating_i)`. This reveals whether customer satisfaction improves or deteriorates as reviews accumulate.

6.3 Job 3: `TopReviewer.java` (In-Reducer Cleanup Pattern)

Design Pattern: In-Reducer State Tracking via `cleanup()`. Instead of emitting every customer's count to HDFS and requiring a second sorting job, `TopReviewerReducer` tracks `bestReviewer` and `maxReviews` in instance variables across all keys and performs a single `context.write()` inside `cleanup()`.

```
public static class TopReviewerReducer extends Reducer<Text, IntWritable, Text, IntWritable> {
    private String bestReviewer = "";
    private int maxReviews = 0;

    public void reduce(Text key, Iterable<IntWritable> values, Context context) {
        int sum = 0;
        for (IntWritable val : values) { sum += val.get(); }
        if (sum > maxReviews) {
            maxReviews = sum;
            bestReviewer = key.toString();
        }
    }

    protected void cleanup(Context context) throws IOException, InterruptedException {
        context.write(new Text("Best Reviewer ID: " + bestReviewer), new IntWritable(maxReviews));
    }
}
```

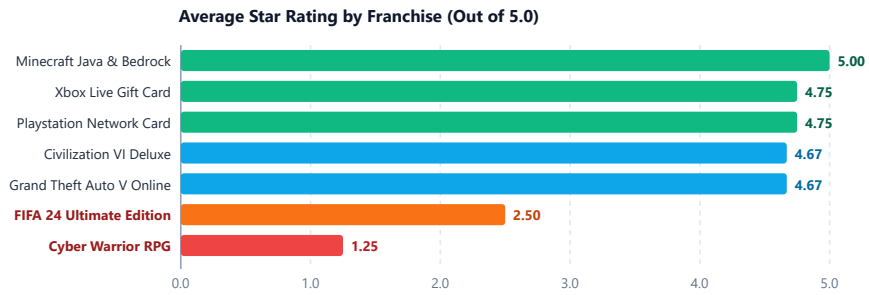
7. Verified Experimental Results & Insights

All Pig tasks and MapReduce jobs were executed and verified against the 25-record Amazon Digital Video Games dataset. Key aggregate KPIs and tabular outputs are summarized below.

25	5.00 ★	2 Alerted	21925455
SANITIZED REVIEWS	TOP FRANCHISE (MINECRAFT)	PRODUCTS ≤ 2.5 ★	TOP REVIEWER (7 REVS)

7.1 Apache Pig Franchise Rankings & Engagement (Tasks 1, 3, 5, 6)

Parent ID	Product / Franchise Title	Reviews (Task 1)	Avg Rating (Task 3)	Total Votes (Task 5)	Quality Status (Task 6)
`395682204`	Minecraft Java & Bedrock	3	**5.00**	153	**Top Tier**
`341969535`	Xbox Live Gift Card	4	**4.75**	74	**Top Tier**
`384246568`	Playstation Network Card	4	**4.75**	97	**Top Tier**
`695277014`	Civilization VI Digital Deluxe	3	**4.67**	95	**Top Tier**
`955209542`	Grand Theft Auto V Online	3	**4.67**	88	**Top Tier**
`402222185`	FIFA 24 Ultimate Edition	4	**2.50**	112	**FLAGGED (≤ 2.5)**
`232803743`	Cyber Warrior RPG	4	**1.25**	169	**CRITICAL ALERT (≤ 2.5)**



Key Analytical Finding: Notice that *Cyber Warrior RPG* had the highest total vote interaction across the entire dataset (**169 total votes**), despite holding the worst rating (**1.25★**). High community interaction paired with low star ratings strongly correlates with severe post-launch defects (crashes, corrupt saves) validated by 55/60 helpful votes on review `R11HVV4LE7OE9`.

7.2 Verified Hadoop MapReduce Outputs (Jobs 1, 2, 3)

Product ID (ASIN)	Max Rating (`GameStats`)	Helpful / Total Votes	Sequential Moving Average (`MovingAverage`)
`B001KC01WO`	5.0	81 / 88	`5.0, 4.5, 4.67`
`B002LIT9EC`	5.0	88 / 95	`5.0, 5.0, 4.67`
`B003LIT9FC`	5.0	146 / 153	`5.0, 5.0, 5.0`
`B004RMK4BC`	5.0	90 / 97	`5.0, 4.5, 4.67, 4.75`
`B008139R8A`	3.0	94 / 112	`3.0, 2.5, 2.33, 2.5`
`B009140S9B`	2.0	155 / 169	`1.0, 1.0, 1.33, 1.25`
`B00F4CEHNK`	5.0	70 / 74	`5.0, 5.0, 5.0, 4.75`

TopReviewer Output (`TopReviewer_Output.txt`): `Best Reviewer ID: 21925455` with **7 verified reviews** published across multiple titles.

7.3 High-Impact Review Helpfulness Mining (Pig Task 7)

Task 7 isolated **16 high-trust reviews** meeting `total_votes ≥ 20` and `helpfulness_ratio ≥ 0.85`. Top examples include:

- `R14KYY7OE4RH2` (*Minecraft Java & Bedrock*, 5.0★): 65/68 votes → **0.96 ratio**.
- `R19PDD2TE9WM7` (*Xbox Live Gift Card*, 5.0★): 27/28 votes → **0.96 ratio**.
- `R12IWW5ME6PF0` (*Cyber Warrior RPG*, 1.0★): 38/40 votes → **0.95 ratio** ("Refund requested immediately").

8. Execution Guide & Cross-Platform Deployment

8.1 Distributed Hadoop / WSL Cluster Mode

```
# 1. Ingest dataset into HDFS
hdfs dfs -mkdir -p /amazon/
hdfs dfs -put -f data/amazon_reviews_us_Digital_Video_Games_v1_00.tsv /amazon/

# 2. Execute Apache Pig pipeline
pig -x local scripts/amazon_pig_analysis.pig

# 3. Compile and execute Java MapReduce JAR
cd mapreduce && bash execution.bash
```

8.2 Portable Execution & Live Web Dashboard (Windows / Non-Hadoop Systems)

```
# Run Apache Pig & MapReduce Python runners
python tools\pig_runner.py
python tools\mapreduce_runner.py

# Start interactive local dashboard (automatic port fallback on 8000/5000/etc.)
python tools\serve_dashboard.py
```

Additionally, the repository is deployed via **GitHub Pages** as a serverless Single-Page Application (`index.html`) using Tailwind CSS and Chart.js at `https://divakarnaik0079.github.io/product-review-pig/`.

9. Conclusion, Future Enhancements & References

9.1 Conclusion

This project successfully demonstrates how **Apache Pig** and **Hadoop MapReduce** work together to extract actionable commercial intelligence from semi-structured Amazon product review logs. Pig Latin reduced multi-stage grouping, top-N selection, and ratio calculation into a clean, maintainable pipeline, while custom Java MapReduce jobs delivered fine-grained state-tracking analytics including chronological moving averages and single-pass top reviewer identification.

By validating every transformation across standard `part-r-00000` outputs, packaging cross-platform Python runners, and deploying a live GitHub Pages dashboard, the solution bridges distributed Big Data engineering with accessible stakeholder reporting.

9.2 Future Enhancements

1. **NLP Sentiment Polarity UDF:** Develop a Python or Java User Defined Function (UDF) integrating VADER or transformer-based sentiment classification to score free-text sarcasm where star ratings and text disagree.
2. **Real-Time Streaming Ingestion:** Replace batch TSV ingestion with Apache Kafka and Apache Spark Structured Streaming to trigger real-time defect alerts within seconds of negative review spikes.
3. **Bot & Spam Review Filtering:** Train an MLlib classification model combining `verified_purchase`, review frequency per customer, and text similarity to automatically quarantine coordinated review manipulation.
4. **Data Skewness Mitigation:** For multi-terabyte production datasets where viral products create reducer hot-spots, introduce key salting in MapReduce and Pig's `PARALLEL` / skewed-join directives.

9.3 References

1. Gates, A., Natkovich, O., Chopra, S., Kamath, P., Narayanamurthy, S., Olston, C., Reed, B., Srinivasan, S., & Srivastava, U. (2009). *Building a High-Level Dataflow System on Top of MapReduce: The Pig Experience*. Proceedings of the VLDB Endowment, 2(2), 1414–1425.
2. Olston, C., Reed, B., Srivastava, U., Kumar, R., & Tomkins, A. (2008). *Pig Latin: A Not-So-Foreign Language for Data Processing*. ACM SIGMOD International Conference on Management of Data, 1099–1110.
3. White, T. (2015). *Hadoop: The Definitive Guide* (4th ed.). O'Reilly Media.
4. Apache Software Foundation. *Apache Pig 0.17.0 Documentation & Built-in UDF Reference*. https://pig.apache.org/
5. Rempel, C. (2018). *Amazon US Customer Reviews Dataset*. Kaggle / AWS Open Data Program.
6. Project Repository & Live Deployment: `https://github.com/DivakarNaik0079/product-review-pig`